



Manual do sistema

Instalação, operação e o que observar
durante a avaliação

PROJETO

Tech Challenge FIAP, Fase 5

ESCOPO

Agente de IA, memória e RAG, API e interface

DOCUMENTO

Versão 1.0

AMBIENTE

Windows, Python 3.11+, Node 20+

Como ler este manual

Este sistema conversa com pessoas usando um modelo de linguagem, e isso tem uma consequência prática que precisa ficar clara antes de qualquer outra coisa.

A REGRA QUE EVITA O FALSO DEFEITO

O texto que a IA escreve muda a cada execução. O comportamento observável do sistema não muda.

Se você mandar “quero comprar um apartamento” duas vezes, vai receber duas redações diferentes da mesma pergunta. Uma hora ela pergunta “Qual região você está procurando?”, outra hora “Em qual bairro você gostaria de morar?”. As duas estão corretas, porque as duas perseguem o mesmo campo.

Por isso este manual **não** manda você digitar uma frase e conferir se a resposta bateu palavra por palavra. Ele dá frases de partida, deixa você conversar do seu jeito, e diz **o que observar**: qual etiqueta apareceu, qual campo o painel preencheu, quando o seletor de data surgiu, se os imóveis respeitaram o orçamento.

Quem tenta validar a redação da IA conclui que o sistema está quebrado. Quem observa o comportamento vê que ele está funcionando. A seção 7 detalha exatamente o que olhar.

Caminho mais curto, se você só quer ver funcionando

- 1 Instale as dependências uma vez: **seção 2**.
- 2 Se quiser a IA de verdade, pegue a chave: **seção 3**. Sem ela o sistema roda igual, com respostas mais robóticas.
- 3 Suba os dois terminais: **seção 4**.
- 4 Abra `http://localhost:5173` e converse: **seção 7**.

O resto do manual existe para quando alguma coisa não fizer sentido, e para quem vai avaliar e precisa saber o que procurar.

CONTEÚDO

Sumário

- | | |
|---|---|
| 1 | O que é este sistema |
| 2 | Antes de começar o que instalar |
| 3 | A chave do Google Gemini e como rodar sem ela |
-

4 **Subir o sistema** dois terminais

5 **Conferir que está tudo no ar**

6 **A tela do Chat** o que cada elemento significa

7 **Como conversar com o agente** frases de partida e o que observar

8 **As telas do corretor**

9 **Privacidade, e o que dizer sobre ela**

10 **Os três cenários do desafio**

11 **Quando algo não funciona**

12 **Limites conhecidos**

A **Ficha técnica**

B **Comandos de referência**

C **Roteiro de cinco minutos**

O que é este sistema

1.1 O problema de negócio

Uma imobiliária recebe leads pelo site, pelo portal e pelo WhatsApp. A maior parte deles nunca vira nada, e o corretor não tem como saber quais valem uma ligação sem conversar com todos. O SDR, sigla de *Sales Development Representative*, é a pessoa que faz esse primeiro filtro: descobre o que o cliente quer, qualifica, e entrega ao corretor apenas quem está pronto.

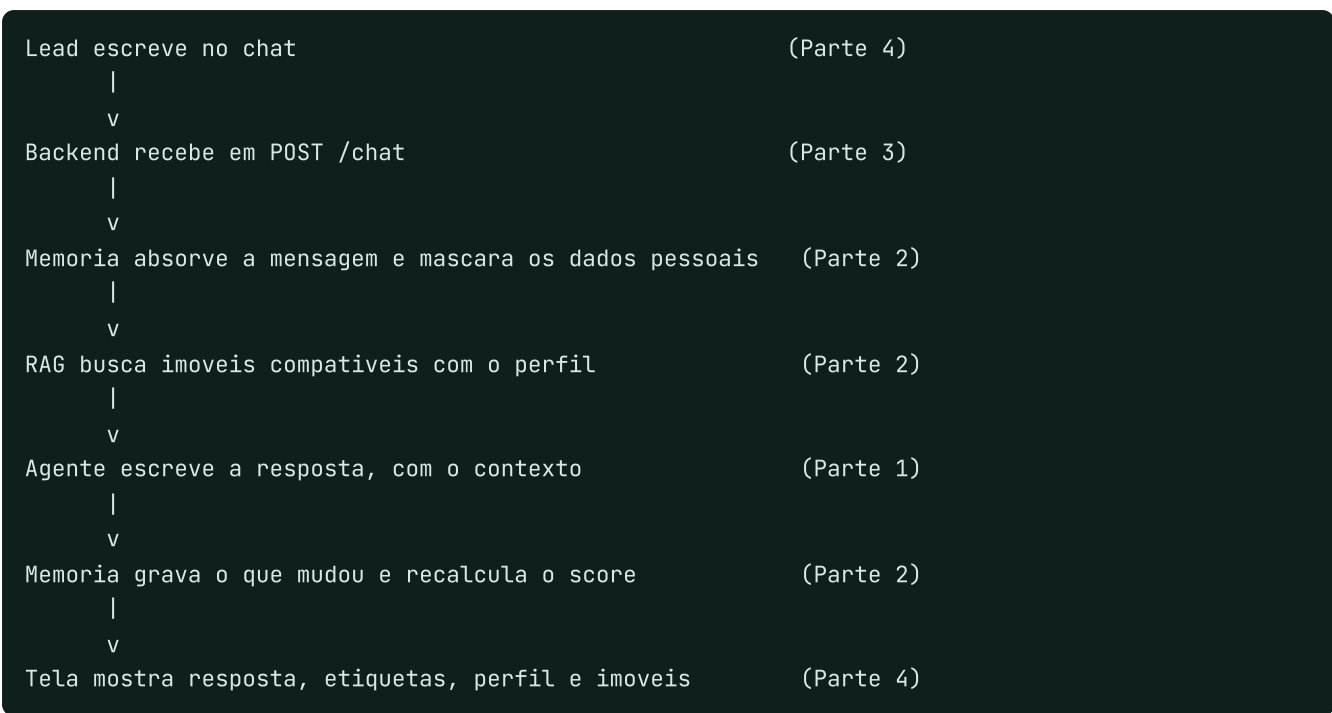
Este projeto é um SDR de software. Ele conversa com o lead, extrai as informações que definem a oportunidade, sugere imóveis do catálogo, marca a visita, e entrega ao corretor uma ficha pronta com nota e próxima ação.

1.2 As quatro partes

O sistema é dividido em quatro módulos, cada um com responsabilidade própria. Eles rodam juntos, mas foram escritos para serem entendidos separadamente.

PARTE	PASTA	DO QUE CUIDA
1. Agente de IA	ai-core/	Conversa com o lead e extrai dados da fala
2. Memória e RAG	ai-memory-rag/	Lembra do lead, busca imóveis, calcula o score, decide o follow-up
3. Backend	backend/	API HTTP, banco de dados, agendamentos, job de retomada
4. Frontend	frontend/	As telas: o chat do lead e o painel do corretor

O caminho de uma única mensagem atravessa as quatro:



1.3 O que você vai ver quando abrir

O sistema tem **duas experiências no mesmo endereço**.

A **tela do lead** (/), que é o chat, é o que um cliente da imobiliária veria. As **telas do corretor** (/painel) são o dashboard com os leads qualificados, a ficha de cada lead, a agenda de visitas e o catálogo de imóveis.

As duas convivem porque isto é uma demonstração acadêmica, e ver os dois lados ao mesmo tempo é justamente o ponto: você escreve no chat e observa o painel reagir.

Antes de começar

2.1 O que precisa estar instalado

PROGRAMA	VERSÃO MÍNIMA	VERSÃO EM QUE FOI TESTADO
Python	3.11	3.11
Node.js	20	24.19
npm	10	11

2.2 Como conferir

Abra o PowerShell e rode, na raiz do projeto:

```
cd "D:\Projects\Pos\Fase 05\Tech-Challenge-FIAP-FASE5"

python --version
node --version
npm --version
```

Você deve ver três linhas com números iguais ou maiores que os da tabela acima.

2.3 Se alguma versão estiver faltando ou velha

Python. Baixe em python.org/downloads e marque a caixa *Add Python to PATH* durante a instalação. Sem essa caixa marcada, o comando `python` não é reconhecido no terminal e você vai precisar digitar o caminho completo toda vez.

Node.js. Baixe a versão LTS em nodejs.org. O npm vem junto, não precisa instalar separado.

DEPOIS DE INSTALAR

Feche e abra o PowerShell de novo. O terminal lê o PATH quando abre, então um terminal que já estava aberto continua sem enxergar o programa recém-instalado.

2.4 Preparar o ambiente Python

O projeto usa um ambiente virtual próprio, para não misturar as bibliotecas dele com as do resto da sua máquina.

```
cd "D:\Projects\Pos\Fase 05\Tech-Challenge-FIAP-FASE5"

python -m venv .venv
.venv\Scripts\python.exe -m pip install --upgrade pip
.venv\Scripts\python.exe -m pip install -r backend\requirements.txt
```

A última linha instala tudo que as quatro partes precisam: FastAPI, uvicorn, SQLAlchemy, Pydantic, APScheduler e o SDK do Google Gemini. A instalação leva alguns minutos. Se aparecer erro de rede, rode de novo: o pip retoma o que já baixou.

POR QUE O CAMINHO COMPLETO

Em todo este manual o Python é chamado como `.venv\Scripts\python.exe`, e não como `python`. Isso é proposital: garante que você está usando o ambiente do projeto, com as bibliotecas certas, mesmo que haja outro Python instalado na máquina.

2.5 Preparar o frontend

```
cd "D:\Projects\Pos\Fase 05\Tech-Challenge-FIAP-FASE5\frontend"

npm install
```

Este comando roda uma vez só, na primeira vez. Ele cria a pasta `node_modules`, que é grande e não vai para o Git.

A chave do Google Gemini

3.1 Você precisa mesmo de uma chave?

Não para rodar. Sim para ver a IA de verdade.

Sem chave, o sistema sobe normalmente e funciona inteiro: o chat responde, a memória grava, os imóveis aparecem, o painel calcula os scores, a agenda funciona. O que muda é quem escreve as respostas. No lugar do Gemini entra um agente simulado, o *mock*, que pergunta de forma direta e robótica o próximo campo que falta no perfil.

Com chave, as respostas são escritas pelo Gemini, e a conversa fica natural.

O cabeçalho da tela avisa em qual modo você está, o tempo inteiro. Não existe o caso de estar em mock sem saber.

RECOMENDAÇÃO PRÁTICA

Desenvolva sem chave, apresente com chave. Assim você não gasta a cota diária ajustando detalhe de tela, e chega na banca com a cota inteira disponível.

3.2 Como obter a chave

- 1 Acesse aistudio.google.com/apikey.
- 2 Entre com uma conta Google.
- 3 Clique em **Create API key**.
- 4 Escolha um projeto do Google Cloud, ou deixe o AI Studio criar um novo.
- 5 Copie a chave. Ela começa com `AIZA` e tem cerca de 39 caracteres.

A chave é gratuita e o plano gratuito não pede cartão de crédito.

TRATE A CHAVE COMO SENHA

Ela dá acesso à sua cota e, com faturamento ativo, à sua fatura. Não cole em print, em slide, em mensagem de grupo nem em commit. O arquivo onde ela mora está no `.gitignore` justamente por isso, e é também por isso que não há uma captura da chave neste manual.

3.3 Onde colar a chave

Crie um arquivo chamado `.env` na **raiz do repositório**, não dentro de `backend/`. A raiz é onde as três partes de Python procuram, então um arquivo só serve para todas.

```
cd "D:\Projects\Pos\Fase 05\Tech-Challenge-FIAP-FASE5"
copy backend\.env.example .env
notepad .env
```

Preencha as duas primeiras variáveis:

```
GEMINI_API_KEY=AIza...sua-chave-aqui
GEMINI_MODEL=gemini-2.5-flash
```

3.4 Por que fixar o modelo

A cota gratuita do Gemini é contada **por modelo**. Se o agente da Parte 1 usasse um modelo e o resumo da Parte 2 usasse outro, o projeto gastaria duas cotas separadas, e você veria um sintoma confuso: o chat parando por erro de cota enquanto o resumo do corretor continua funcionando.

Por isso as duas metades leem a mesma variável. Fixe um id em vez de confiar no padrão do código. Dois modelos foram verificados neste projeto: `gemini-2.5-flash`, que é o padrão, e `gemini-3.6-flash`.

3.5 O que é a cota gratuita, na prática

O plano gratuito dá **20 requisições por dia, por modelo**.

Cada mensagem que você manda no chat consome **uma** requisição de geração. Uma conversa de qualificação completa tem de seis a oito mensagens, então você consegue rodar **duas ou três conversas inteiras por dia** na cota gratuita.

Isso é pouco, e é bom saber antes, não durante a apresentação. O sistema tem três defesas:

- 1 **A busca de imóveis usa outro modelo** (`gemini-embedding-001`), que tem cota própria, e o índice fica em cache no disco. Ela não gasta a sua cota de conversa.
- 2 **O job de follow-up não usa IA por padrão**. Ele escreve o texto por regra. Antes ele gastava a mesma cota do chat em segundo plano, a cada 30 minutos, e a pessoa via o chat cair em mock sem entender por quê.
- 3 **Quando a cota acaba, o chat não quebra**. Ele cai no agente mock, marca a bolha na tela, e a qualificação continua andando.

Se precisar de mais volume, habilite faturamento no AI Studio, em *Projects, Set up billing*. Com faturamento ativo, o custo do `gemini-2.5-flash` no volume deste projeto fica na casa de centavos por apresentação.

3.6 Conferir se a chave está funcionando

O projeto tem uma ferramenta que traduz o erro do Gemini em **o que fazer**. Isso importa porque a API devolve mensagens parecidas, como “429” e “403”, para situações bem diferentes: cota do dia esgotada,

crédito zerado, projeto bloqueado, tipo de chave errado. Cada uma pede uma ação diferente, e adivinhar custa uma tarde.

```
cd "D:\Projects\Pos\Fase 05\Tech-Challenge-FIAP-FASE5"
.venv\Scripts\python.exe docs\verificar-chave.py
```

Saída quando está tudo certo:

Chave e modelo configurados no .env da raiz

```
chave: AIzaSyAb... (39 caracteres)
modelo: gemini-2.5-flash
```

```
ok    geracao (gemini-2.5-flash)    1.8s
ok    embedding (RAG)              0.9s
```

Tudo certo. As duas metades do sistema vão usar gemini-2.5-flash.

O script nunca imprime a chave inteira, apenas o prefixo e o tamanho. Quando algo falha, ele diz o motivo e o caminho:

```
FALHA geracao (gemini-2.5-flash)    429 RESOURCE_EXHAUSTED ...
→ Cota GRATUITA do dia acabada para este modelo (sao 20 por
  dia e por modelo). Espere amanha, troque o GEMINI_MODEL,
  ou habilite faturamento.
```

Subir o sistema

O sistema precisa de **dois terminais abertos ao mesmo tempo**. Um roda a API, outro roda a interface. Os dois ficam ocupados enquanto o sistema estiver no ar.

4.1 Terminal 1: o backend

```
cd "D:\Projects\Pos\Fase 05\Tech-Challenge-FIAP-FASE5"
.venv\Scripts\python.exe backend\run.py
```

Quando dá certo, você vê algo próximo disto:

```
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:      Started server process [33756]
INFO:      Application startup complete.
[followup] Ativo, a cada 30 min.
[api] Pronta. Docs em /docs
[rag] 140 imóveis indexados (gemini-embedding-001, cache).
```

A última linha aparece alguns segundos depois das outras, porque o índice de busca é construído numa thread separada para não atrasar a subida da API. Três variações normais nela:

O QUE APARECE	O QUE SIGNIFICA
(cache)	O índice veio do disco, sem custo. É o caso comum
(rebuild)	Ele foi construído agora. Acontece na primeira execução e ao trocar de modelo de embedding
hashing-v1	Você está sem chave, e a busca usa o modo lexical. Funciona, com resultados um pouco menos precisos

4.2 Terminal 2: o frontend

Abra um **segundo** PowerShell, sem fechar o primeiro.

```
cd "D:\Projects\Pos\Fase 05\Tech-Challenge-FIAP-FASE5\frontend"
npm run dev
```

Quando dá certo:

```
VITE v7.3.6 ready in 412 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
```

4.3 Os três endereços

ENDEREÇO	O QUE É
<code>localhost:5173</code>	A aplicação. É aqui que você trabalha
<code>localhost:8000/docs</code>	Documentação interativa da API, gerada pelo FastAPI
<code>localhost:8000/health</code>	Diagnóstico em JSON do que está ligado

O `/docs` vale uma visita durante a avaliação: ele lista todas as rotas e deixa executar cada uma pelo navegador, sem precisar de ferramenta externa.

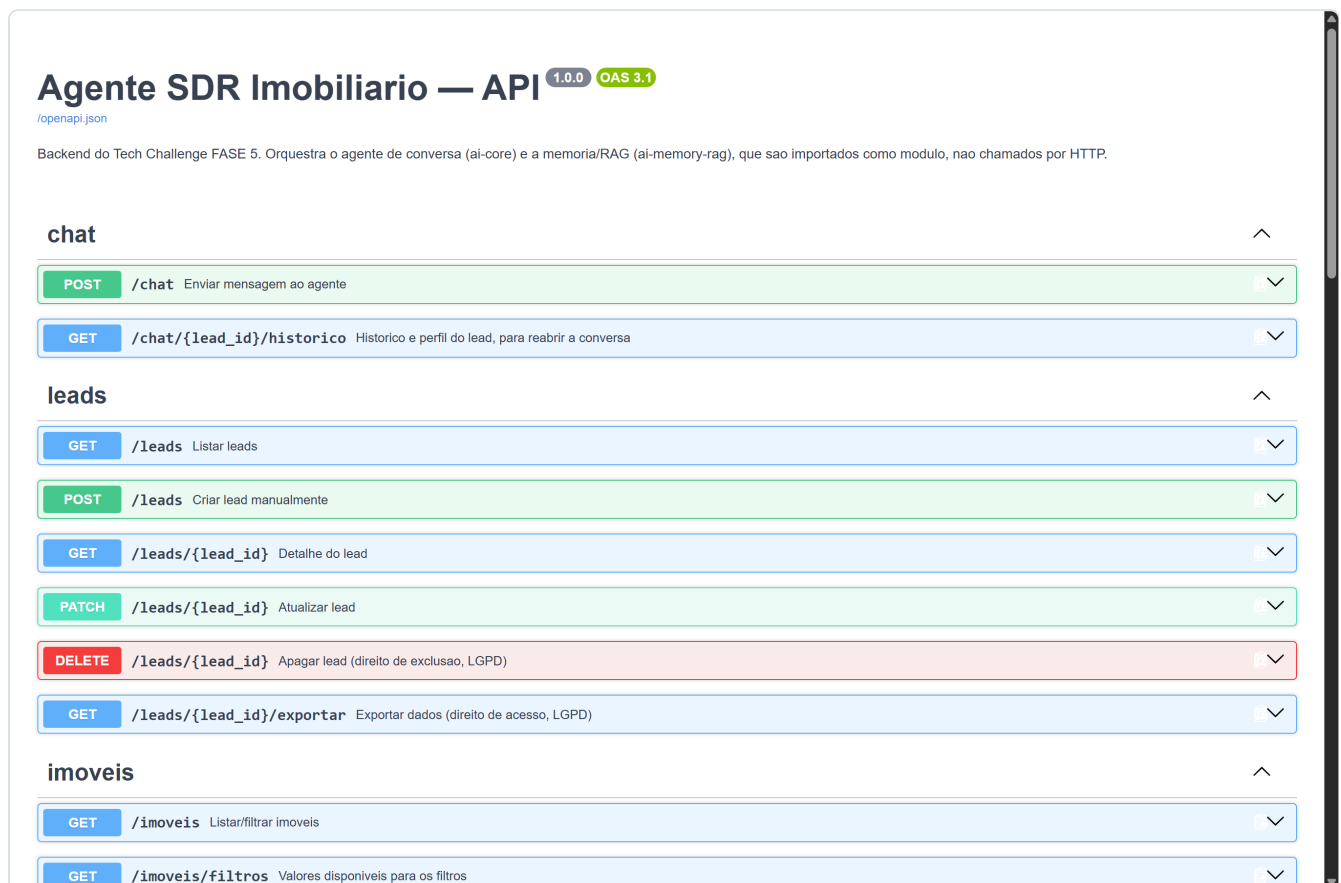


Figura 2. A documentação interativa em `/docs`, com as rotas agrupadas por assunto. Cada uma abre e pode ser executada ali mesmo.

4.4 Se a porta 5173 estiver ocupada

O Vite recusa subir se a porta já estiver em uso, e a mensagem é clara: `Error: Port 5173 is already in use`. Quase sempre isso significa que existe outro `npm run dev` rodando, de uma sessão anterior que não foi fechada. Duas saídas:

```
# usar a outra porta ja liberada no CORS
npm run dev:3000
```

ou encerrar o processo antigo:

```
Get-NetTCPConnection -LocalPort 5173 |  
  Select-Object -ExpandProperty OwningProcess |  
  ForEach-Object { Stop-Process -Id $_ -Force }
```

SÓ 5173 E 3000 ESTÃO LIBERADAS

Essas são as duas portas no CORS do backend. Qualquer outra precisa ser acrescentada em `CORS_ORIGINS` no `.env` da raiz, e sem isso toda chamada morre com um erro que não parece ser de porta.

4.5 No dia da apresentação, use `--no-reload`

```
.venv\Scripts\python.exe backend\run.py --no-reload
```

O autoreload é ótimo enquanto se escreve código: ele reinicia a API sozinho a cada arquivo salvo. Na hora de apresentar ele atrapalha, por dois motivos. Primeiro, salvar um arquivo sem querer reinicia o backend no meio de uma frase do agente. Segundo, cada reinício paga uma consulta de sondagem ao serviço de embedding antes de responder a primeira mensagem.

Conferir que está tudo no ar

5.1 O cabeçalho, que é o painel de instrumentos

O canto direito do cabeçalho carrega o estado do sistema, e ele se atualiza sozinho: a cada minuto quando está tudo bem, a cada dez segundos quando a API está fora. Você não precisa recarregar a página para ver o estado mudar.

SELO	COR	SIGNIFICA
API no ar	verde	O backend respondeu
API offline	vermelho	O backend não respondeu. O terminal 1 caiu, ou nunca subiu
IA em modo mock	âmbar	As respostas vêm do agente simulado, não do Gemini
nenhum selo	cinza	Ainda verificando. Aparece por um instante no carregamento

O selo âmbar de mock aparece **junto** com o verde de API no ar. Eles dizem coisas diferentes: um é sobre o backend estar respondendo, outro é sobre quem está escrevendo as respostas.



Figura 3. O cabeçalho rodando sem chave: o selo verde diz que a API respondeu, e o âmbar diz que quem escreveu a resposta foi o agente simulado. À direita, o seletor de tema com os três estados.

5.2 O `/health`, que é o diagnóstico completo

Abra `http://localhost:8000/health` no navegador:

```
{
  "status": "ok",
  "banco": "ok",
  "database_url": "D:\\...\\backend\\data\\app.db",
  "ia": {
    "parte2": true,
    "agente": "gemini",
    "modo_configurado": "gemini",
    "llm": "gemini-2.5-flash",
    "rag": {
      "carregado": true,
      "imoveis": 140,
      "embedder": "gemini-embedding-001",
      "ultima_busca_ok": true
    },
    "detalhe": ""
  }
}
```

CAMPO	O QUE DIZ
<code>status</code>	A API está de pé
<code>banco</code>	O banco respondeu a uma consulta de teste
<code>database_url</code>	Qual arquivo de banco está em uso. Útil quando “sumiram os dados”
<code>ia.parte2</code>	Se a memória e o RAG foram importados com sucesso
<code>ia.agente</code>	Quem respondeu o último turno de verdade
<code>ia.modos_configurado</code>	O que foi configurado na subida
<code>ia.llm</code>	O modelo em uso
<code>ia.rag.carregado</code>	Se o índice de imóveis está em memória
<code>ia.rag.imoveis</code>	Quantos imóveis foram indexados. Deve ser 140
<code>ia.rag.embedder</code>	<code>gemini-embedding-001</code> com chave, <code>hashing-v1</code> sem
<code>ia.rag.ultima_busca_ok</code>	Se a última busca por imóvel funcionou. <code>null</code> antes da primeira
<code>ia.detalhe</code>	Explicação em texto quando algo está degradado

A diferença entre `agente` e `modo_configurado`

Esses dois campos existem por causa de um problema real: o modo era decidido uma vez, na subida, e nunca mais. Com chave inválida, projeto bloqueado ou cota estourada, o sistema continuava anunciando “gemini” enquanto **todo** turno era respondido pelo mock, e o selo do cabeçalho ficava verde mentindo.

Hoje, `modo_configurado` responde “o que eu tentei ser quando subi”, e `agente` responde “o que eu consegui ser no último turno”. Quando os dois divergem, o campo `detalhe` explica e o selo âmbar aparece na tela.

SE VOCÊ VIR GEMINI CONFIGURADO E MOCK ACONTECENDO

A causa está entre chave inválida, projeto bloqueado e cota do dia esgotada. O `docs\verificar-chave.py` da seção 3.6 diz qual das três em menos de dois segundos.

A tela do Chat

Esta é a tela que um cliente da imobiliária veria. Ela tem seis elementos, e cada um mostra uma parte diferente do que o sistema está fazendo.

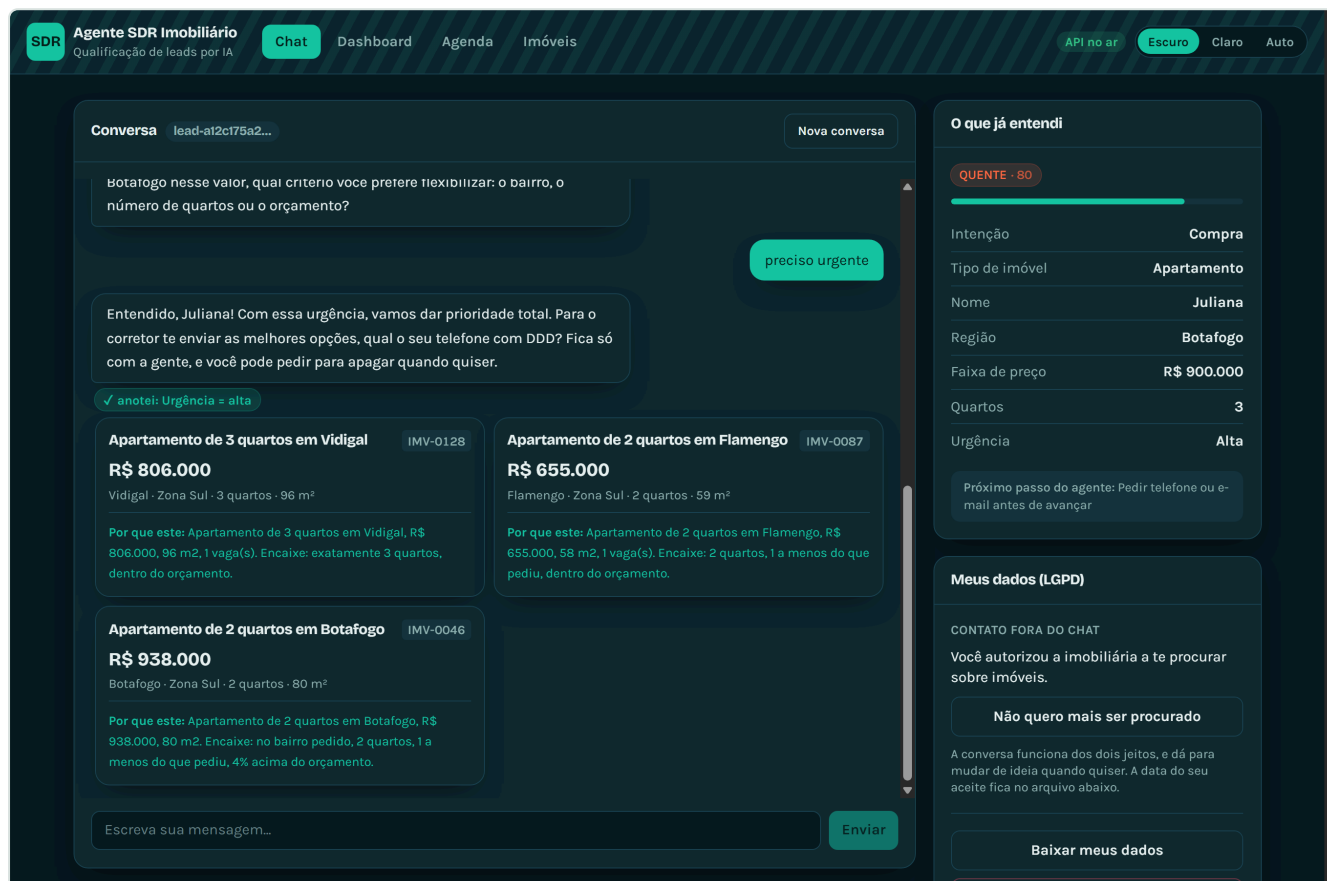


Figura 4. A tela depois de algumas mensagens. Abaixo da resposta, a etiqueta verde de “anotei” e os três imóveis sugeridos, cada um com o seu “Por que este”. À direita, o painel do perfil já com temperatura, nota e seis campos preenchidos.

6.1 A conversa

A coluna da esquerda. As mensagens do lead ficam à direita, em verde. As do agente ficam à esquerda, em cinza. Quando o agente está pensando, aparecem três pontinhos animados.

Quando a resposta veio do agente simulado, a bolha traz uma marca discreta escrita **mock** ao lado. A tela nunca esconde quem respondeu.

6.2 As etiquetas de “anotei”

Logo abaixo de cada resposta do agente aparecem etiquetas verdes, uma por campo que a memória gravou naquele turno.

✓ anotei: Telefone = 21 96655-8899

Figura 5. A etiqueta é a prova visível de que a extração funcionou. Quando o sistema *corrige* um dado que já tinha, ela muda de palavra e passa a dizer “corrigi”.

POR QUE AS ETIQUETAS IMPORTAM TANTO

Elas são o principal ponto de observação de toda a demonstração, porque **não dependem da redação da IA**. Ou o campo foi extraído, ou não foi.

6.3 O painel “O que já entendi”

A coluna da direita, no alto. É o perfil do lead como o sistema o enxerga naquele instante, com os rótulos em português. Ele mostra também o status da qualificação, a nota e a temperatura.

Esses três números são os mesmos que o corretor vai ver no painel dele: não há cálculo diferente para as duas telas. E eles voltam preenchidos depois de um F5, o que é a prova de que a memória é do servidor, não da aba do navegador.

6.4 Os cards de imóvel

Quando o sistema já sabe o suficiente, ou seja, região, quartos ou faixa de preço, ele busca no catálogo de 140 imóveis e mostra até três sugestões abaixo da resposta.

Cada card traz o título, o bairro, o preço e, o que mais importa, **o porquê daquela sugestão**: quantos quartos, quanto do orçamento ocupa, e se a região ou o valor foram relaxados para achar alternativa.

Esse “porquê” é o que separa uma lista de imóveis de uma recomendação. Ele é montado pela Parte 2 a partir dos filtros que foram realmente aplicados, e não escrito pela IA, então ele é sempre verdadeiro em relação à busca.

6.5 O seletor de data

Quando o perfil fica completo, o agente convida para agendar e **um seletor de data aparece sozinho na tela**, abaixo da conversa.

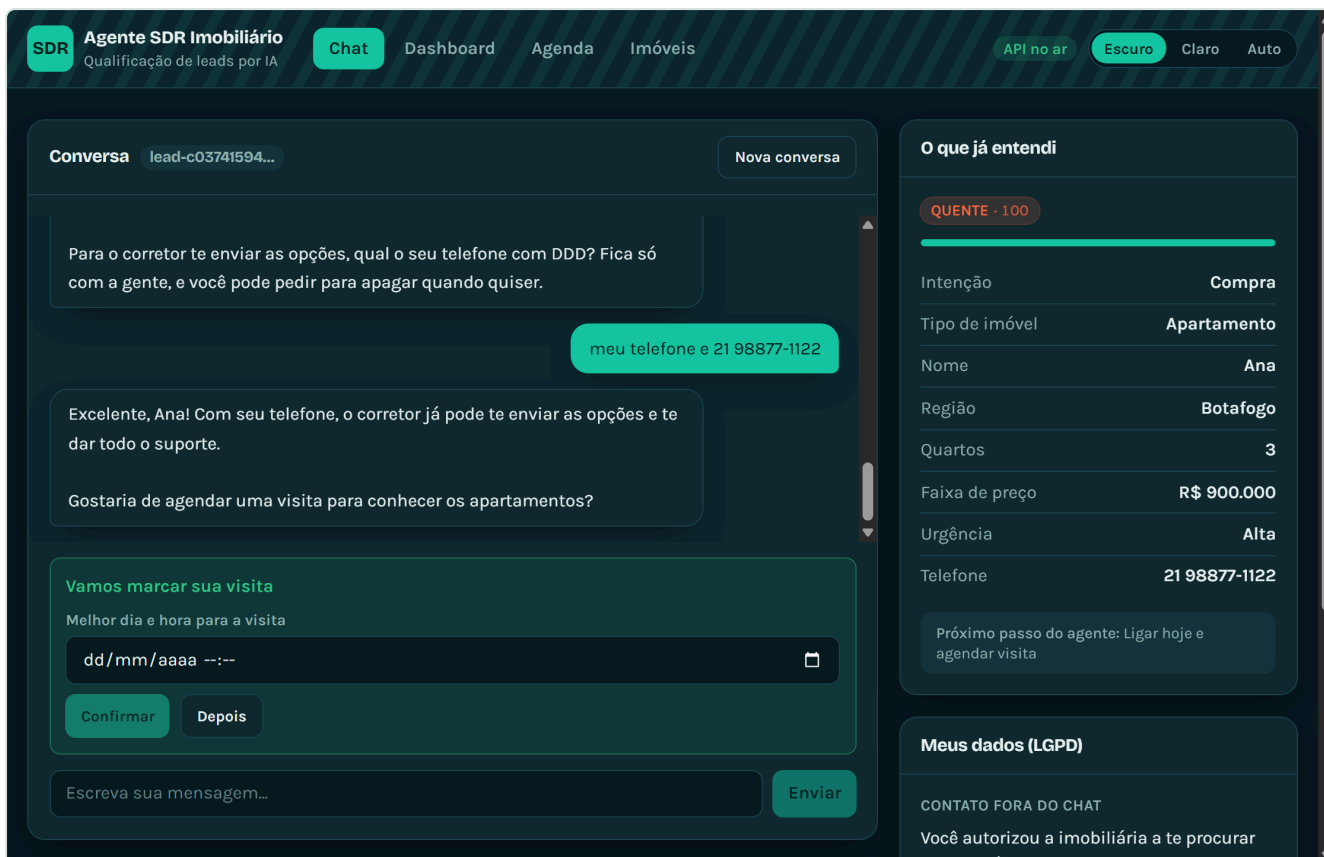


Figura 6. O fim de uma qualificação completa. O agente pediu o telefone, convidou para agendar, e o seletor abriu sozinho abaixo da conversa. O título muda conforme a intenção: “sua visita” para quem compra ou aluga, “sua conversa com o especialista” para quem investe. À direita, o perfil fechado em **QUENTE, 100 pontos**.

A REGRA DO AGENDAMENTO

O agente nunca pergunta dia e hora por escrito. Quem resolve isso é o seletor. Se o agente perguntasse em texto, o lead escreveria uma data que não viraria agendamento nenhum, e teria que digitar tudo de novo no formulário.

O seletor recusa datas no passado, e os horários andam de trinta em trinta minutos. Se você clicar em **Depois**, ele fecha e não volta a abrir sozinho, para não insistir a cada turno.

6.6 O card “Meus dados (LGPD)”

Telefone

21 98877-1122

Próximo passo do agente: Ligar hoje e agendar visita

Meus dados (LGPD)

CONTATO FORA DO CHAT

Você autorizou a imobiliária a te procurar sobre imóveis.

Não quero mais ser procurado

A conversa funciona dos dois jeitos, e dá para mudar de ideia quando quiser. A data do seu aceite fica no arquivo abaixo.

Figura 7. Os três controles que o lead tem sobre os próprios dados, sem precisar pedir a ninguém.

CONTROLE	O QUE FAZ
Pode me procurar / Não quero mais ser procurado	Liga e desliga o follow-up automático, sem apagar a conversa
Baixar meus dados	Gera um JSON com tudo que o sistema guarda daquele lead
Excluir meus dados	Apaga o lead, as mensagens, os agendamentos e a memória da IA

Repare que revogar o contato **não pede confirmação**, e excluir **pede**. Isso não é inconsistência. Uma caixa de diálogo no caminho de quem quer sair é atrito colocado de propósito, e é justamente o que a LGPD não admite. Já excluir é destrutivo e irreversível, então confirmar é proteger o usuário.

Como conversar com o agente

Esta é a seção central do manual.

7.1 A regra que evita o falso defeito

O agente é um modelo de linguagem. **A redação dele varia a cada execução**, mesmo com a mesma entrada. Ele pode cumprimentar, elogiar a escolha do bairro, juntar duas perguntas numa frase, inverter a ordem de duas frases. Nada disso é defeito.

O que **não** varia, porque não depende do modelo:

- Quais campos o sistema persegue, e em que ordem
- Quais etiquetas de “anotei” aparecem
- Quais campos o painel do perfil preenche
- Quando o seletor de data surge
- Se os imóveis sugeridos respeitam o orçamento informado
- Se o investidor recebe consultoria em vez de visita
- Quanto o lead pontua e qual temperatura ele recebe

Essas sete coisas são decididas por código, não pelo modelo. São elas que você deve observar.

7.2 As três frases de partida

A **intenção** é o primeiro campo da ordem de coleta, e é ela que decide todo o resto do funil. Por isso vale começar por uma frase que deixe a intenção clara. Dali em diante, responda naturalmente, do seu jeito.

CAMINHO	FRASE DE PARTIDA
Compra	Oi, quero comprar um apartamento
Aluguel	Oi, estou procurando um apartamento para alugar
Investimento	Oi, quero investir em imóveis para alugar

REPARE NA TERCEIRA

Ela tem a palavra “alugar” dentro, e mesmo assim a intenção correta é **investimento**. Ali, “alugar” diz o que a pessoa vai fazer com o imóvel, não por que ela está procurando. Quem procura para morar não fala em investir. Vale observar essa etiqueta com atenção.

7.3 O que o agente vai perseguir, por caminho

As ordens de coleta são **diferentes** por intenção, e essa diferença é uma das coisas mais importantes para observar.

Comprador e locatário

ORDEM	CAMPO	COMO A PERGUNTA COSTUMA SOAR
1	Intenção	comprar, alugar ou investir?
2	Região	qual bairro ou região?
3	Quartos	quantos quartos?
4	Faixa de preço	qual orçamento, ou valor do aluguel?
5	Urgência	para quando precisa?
6	Telefone	qual o telefone com DDD?

Investidor

ORDEM	CAMPO	COMO A PERGUNTA COSTUMA SOAR
1	Intenção	confirmar que é investimento
2	Ticket de investimento	quanto pretende aplicar?
3	Retorno esperado	quanto espera receber, em % ao ano ou R\$ por mês?
4	Região	qual bairro ou região?
5	Urgência	para quando pretende investir?
6	Telefone	qual o telefone com DDD?

O INVESTIDOR NUNCA É PERGUNTADO SOBRE QUARTOS

Quem investe decide por ticket e retorno, e perguntar quantos quartos ele quer é a pergunta errada, que queima a credibilidade da conversa. Se você vir o agente perguntando quartos a um investidor, isso é um defeito, e vale reportar.

Nome e e-mail não são perseguidos por nenhum dos dois caminhos. Se você se apresentar, o agente usa o nome. Se não, ele não cobra.

7.4 O que observar durante a conversa

Use esta lista como roteiro. Ela vale para qualquer redação que a IA escolher.

A cada resposta

- ☐ A etiqueta de “anotei” apareceu para o dado que você acabou de informar
- ☐ O painel “O que já entendi” ganhou aquele campo, com o rótulo em português

- ☐ O agente não repetiu uma pergunta que você já respondeu
- ☐ O agente fez uma pergunta por vez, e não despejou um questionário

Quando você informar a região ou os quartos

- ☐ Cards de imóvel apareceram abaixo da resposta
- ☐ Cada card explica por que aquele imóvel foi sugerido

Quando você informar o orçamento

- ☐ Os imóveis sugeridos passaram a respeitar o valor
- ☐ Nenhum imóvel muito acima do orçamento aparece sem justificativa no card

Quando o perfil completar

- ☐ O seletor de data apareceu sozinho, sem você clicar em nada
- ☐ O agente convidou para agendar, e não perguntou dia e hora em texto
- ☐ Para investidor, o convite fala em **especialista** ou **consultoria**
- ☐ Para comprador e locatário, o convite fala em **visita**

No painel do perfil, a qualquer momento

- ☐ A temperatura e a nota subiram conforme você informou mais dados
- ☐ Para investidor, aparecem **Ticket** e **Retorno esperado**, e não Quartos

7.5 Coisas que valem provocar de propósito

Estas situações mostram decisões de projeto que não aparecem numa conversa tranquila.

ESCREVA ISTO	E OBSERVE
550 quando ele perguntar quantos quartos	O perfil não grava 550, e o agente pede de novo. Um número implausível é recusado porque um painel com dado inventado é pior que um painel com campo vazio
R\$ 8.550 na pergunta de quartos	O valor vira orçamento , não quartos. A forma do que você escreveu manda mais que a pergunta que estava em aberto
99999-4410, telefone sem DDD	O agente pede o DDD, e pede só o DDD, sem repetir a pergunta inteira
Diga Tijuca, e três mensagens depois diga Botafogo	A etiqueta muda para corrigi , e o painel troca o valor
não quero mais receber mensagens	Isso bloqueia o follow-up automático para sempre, mesmo com o consentimento marcado
quero uma casa em vez de apartamento	O tipo entra no perfil e a busca passa a trazer casas. Sala comercial deixa de aparecer para quem falou em quartos

7.6 Quando a resposta parecer estranha

Antes de concluir que é defeito, confira três coisas:

- 1 **O selo do cabeçalho está âmbar?** Se sim, quem respondeu foi o mock, e ele é robótico de propósito.
- 2 **A etiqueta apareceu?** Se apareceu, a extração funcionou, e o que você está avaliando é redação, que varia.
- 3 **O painel preencheu o campo?** Se preencheu, o sistema entendeu, mesmo que a frase da IA tenha ficado esquisita.

Se as três respostas forem “sim, funcionou” e a frase ainda assim estiver ruim, aí sim vale anotar: é um problema de prompt, e o lugar dele é `ai-core/src/agent.py`, na constante `SYSTEM_PROMPT`.

As telas do corretor

8.1 Dashboard

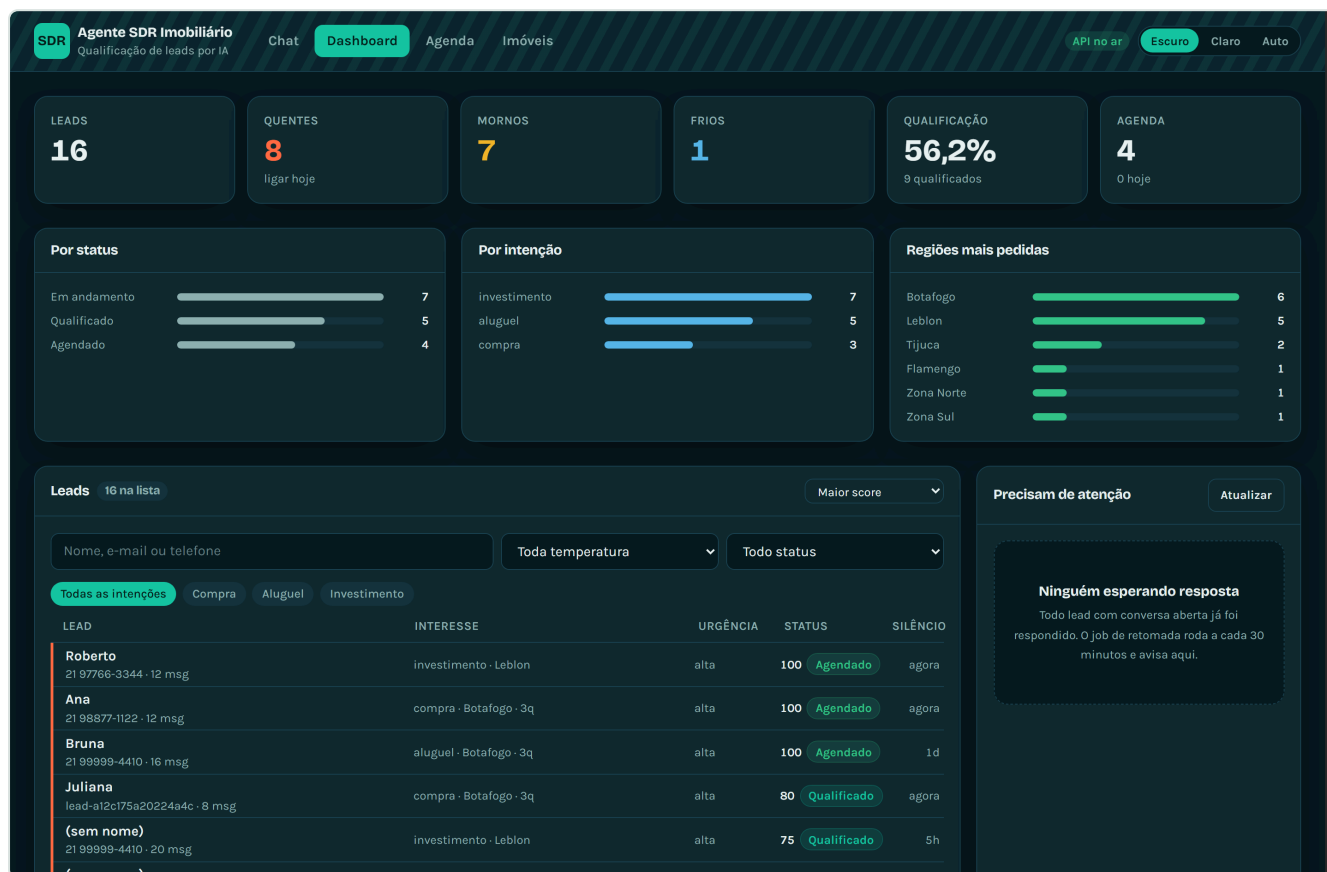


Figura 8. Os seis números do topo, os três gráficos e a lista de leads com a faixa de temperatura na borda esquerda de cada linha. À direita, os leads que estão devendo resposta.

Os seis números do topo

INDICADOR	O QUE CONTA
Leads	Total de leads na base
Quentes	Nota igual ou maior que 70
Mornos	Nota entre 40 e 69
Frios	Nota abaixo de 40
Qualificação	Percentual de leads que chegaram a qualificado
Agenda	Agendamentos próximos, e quantos são hoje

Os três gráficos

Por status mostra em que ponto do funil os leads estão. **Por intenção** mostra a divisão entre compra, aluguel e investimento. **Regiões mais pedidas** mostra a demanda por bairro, e para uma imobiliária

esse é o gráfico que diz onde vale captar imóvel novo.

A lista de leads

Cada linha tem uma faixa colorida na borda esquerda, que é a temperatura. O corretor varre trinta leads pela cor e só lê a linha onde vai parar. A lista tem busca por nome, e-mail ou telefone, filtro por temperatura, status e intenção, e ordenação por nota, silêncio ou data.

A coluna **Silêncio** é quanto tempo faz desde a última mensagem daquele lead. É o número que decide o follow-up.

Precisam de atenção

O cartão da direita lista os leads em silêncio há tempo suficiente para merecer uma retomada, do mais silencioso ao menos. Cada item traz o motivo da decisão em texto, como “72h de silêncio, tentativa 2 de 3”, e um botão que gera o texto sugerido.

O ÚNICO BOTÃO DO PAINEL QUE GASTA COTA

É o de gerar o texto de retomada, e ele só gasta quando você clica. Todo o resto do painel é calculado por regra.

8.2 Detalhe do lead

A interface do sistema 'Agente SDR Imobiliário' apresenta o detalhe de um lead qualificado. No topo, há uma barra de navegação com links para Chat, Dashboard, Agenda e Imóveis, além de opções de tema (Escuro, Claro) e idioma (Auto). O lead em foco é 'Roberto', com o número de telefone 21 97766-3344, classificado como 'QUENTE - 100' e 'Agendado'. Abaixo do nome, há uma barra de status com opções: 'Novo', 'Em andamento', 'Qualificado', 'Agendado' (selecionado) e 'Descartado'. Botões para 'Exportar JSON' e 'Apagar (LGPD)' estão disponíveis. O painel é dividido em seções: 'Resumo para o corretor' com uma recomendação de encaminhamento para o especialista em investimentos, uma descrição do lead (ticket de 2m, urgência alta) e uma lista de imóveis mostrados; 'Perfil qualificado' com detalhes como intenção (investimento), ticket (2m), retorno esperado (12k), região (Leblon) e urgência (alta); e 'Conversa (12 mensagens)' com a última troca de mensagens. Um sidebar à direita mostra 'Agendamentos' com um agendamento para 14/09/2026.

Figura 9. A ficha de um investidor qualificado. O cabeçalho traz temperatura, nota e status; abaixo ficam as ações do corretor e o resumo com a próxima ação recomendada.

Como a nota é calculada

Esta é a parte que a banca costuma perguntar. A nota vai de 0 a 100, e é a soma de sete fatores:

FATOR	PONTOS	QUANDO CONTA
Intenção informada	20	O lead disse se compra, aluga ou investe
Contato disponível	20	Tem telefone ou e-mail
Faixa de preço	15	Para investidor, conta o ticket no lugar
Urgência alta	15	O lead declarou pressa
Região informada	10	
Quartos informados	10	Para investidor, conta o retorno esperado no lugar
Engajado	10	O lead trocou três mensagens ou mais

Contato pesa tanto quanto intenção porque, para um SDR, contato é o produto final: um lead perfeitamente qualificado sem telefone é um lead que o corretor não consegue atender.

A substituição de dois fatores para o investidor existe porque faixa de preço e quartos são exatamente os campos que o funil de investimento não pergunta. Sem essa troca, um investidor que respondesse tudo ficaria com 60 pontos no teto e nunca chegaria a quente.

Desconto por silêncio. Um lead que pontuou 80 e sumiu há um mês não está quente. A nota perde 10 pontos depois de uma semana de silêncio, 25 depois de um mês, e até 40 no limite.

Temperatura: 70 ou mais é QUENTE, de 40 a 69 é MORNNO, abaixo de 40 é FRIO.

A ficha mostra os fatores um a um, e não só o número final, porque o corretor precisa saber **por que** o lead está quente para decidir o que dizer na ligação.

Resumo e próxima ação

Um parágrafo que diz o que o lead quer, o que ele ainda não informou, e qual é a próxima ação recomendada. Ele é gerado por regra, sem gastar IA; existe um botão para gerar a versão escrita pelo Gemini.

SITUAÇÃO	PRÓXIMA AÇÃO
Sem intenção conhecida	Descobrir se é compra, aluguel ou investimento
Sem contato	Pedir telefone ou e-mail antes de avançar
Quente e investidor	Encaminhar para o especialista em investimentos hoje
Quente	Ligar hoje e agendar visita
Morno	Perseguir os campos que faltam, conforme a intenção

Ações do corretor

Mover para muda o status entre novo, em andamento, qualificado, agendado e descartado. **Exportar JSON** baixa tudo que o sistema guarda daquele lead. **Apagar (LGPD)** remove o lead, as mensagens, os agendamentos e a memória da IA; pede confirmação, e não tem volta.

8.3 Agenda

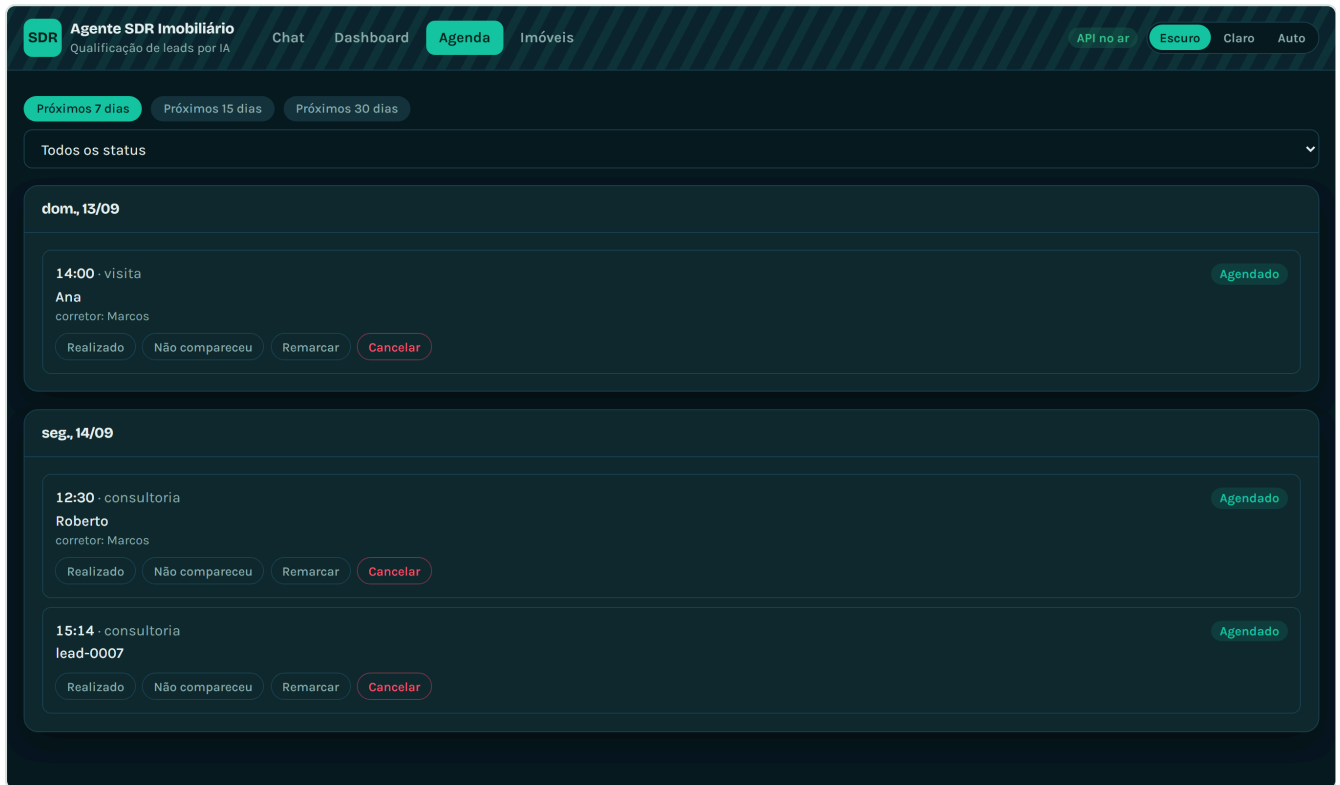


Figura 10. Os agendamentos agrupados por dia, com a janela de tempo e o filtro de status no alto. Cada item tem as quatro ações do corretor.

Cada item traz hora, tipo (visita, reunião ou consultoria), o lead com link para a ficha, e quatro ações: **Realizado**, **Não compareceu**, **Remarcar** e **Cancelar**. Remarcar recusa data no passado, igual ao seletor do chat.

8.4 Imóveis

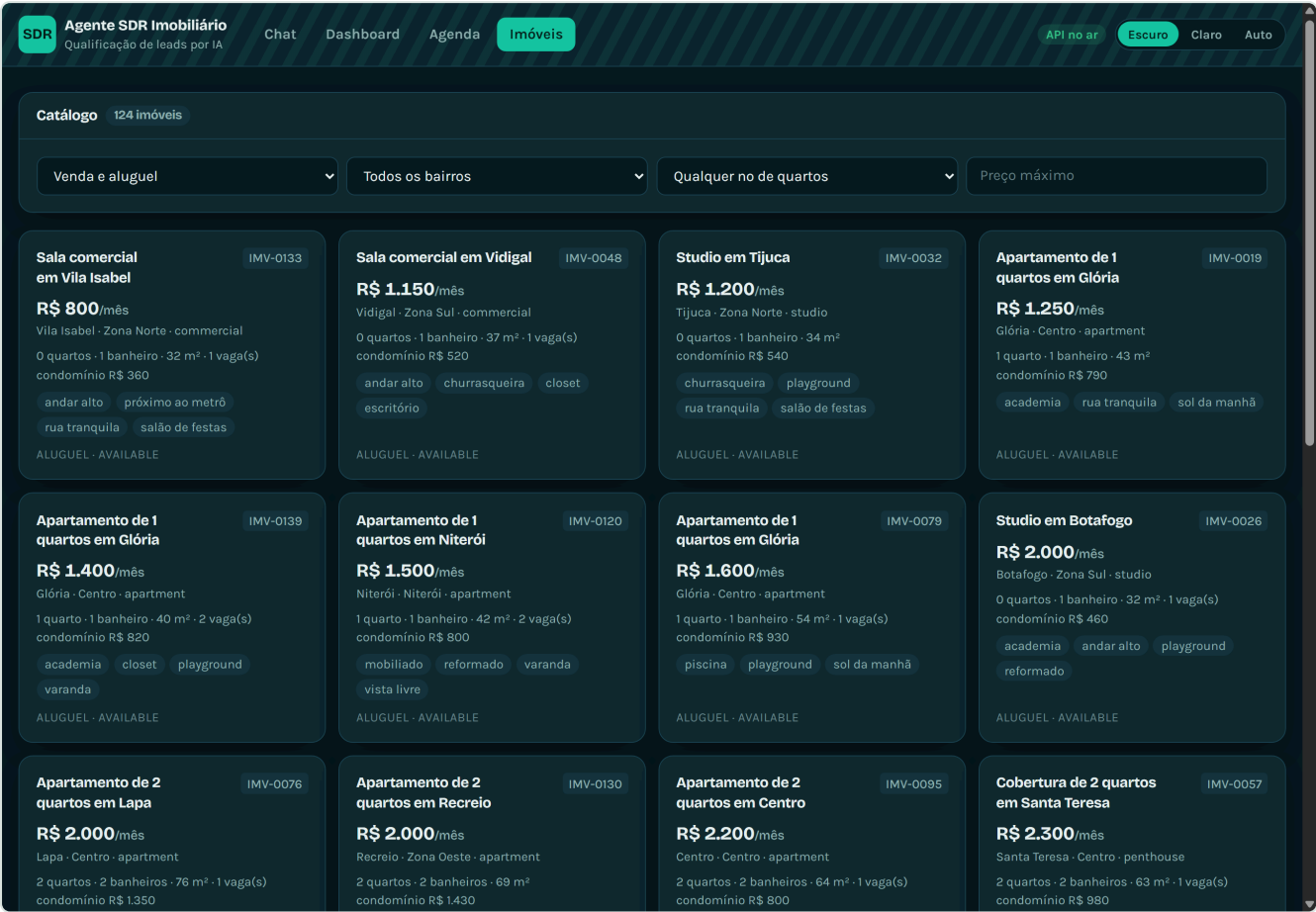


Figura 11. O catálogo que alimenta as sugestões do chat, com filtro por tipo de negócio, bairro, quartos e preço.

São 140 imóveis em 20 bairros do Rio de Janeiro:

RECORTE	QUANTIDADE	RECORTE	QUANTIDADE
À venda	93	Apartamentos	83
Para alugar	47	Casas	20
Coberturas	17	Studios	11
Salas comerciais	9		

DICA PARA A DEMONSTRAÇÃO

Abra esta tela **antes** de conversar no chat, para saber o que existe na base. Assim você pede algo que tem estoque, e as sugestões do chat ficam mais interessantes de mostrar.

Privacidade, e o que dizer sobre ela

O documento completo é o [docs/privacidade-e-limites.md](#). Esta seção é o resumo que serve para responder na apresentação.

9.1 O que sai do sistema

Toda mensagem passa por pseudonimização antes de chegar ao Google Gemini. Nome, e-mail e telefone são trocados por apelidos:

```
ORIGINAL : meu nome e Marcos, email marcos@teste.com,  
          telefone 21 99999-4410, quero 3 quartos em Botafogo  
  
ENVIADO  : meu nome e [NOME_1], email [EMAIL_1],  
          telefone [TELEFONE_1], quero 3 quartos em Botafogo
```

O que interessa para achar imóvel, “3 quartos em Botafogo”, atravessa intacto: a proteção não custa qualidade de resposta.

São três os caminhos que falam com o Gemini, e os três recebem o texto mascarado: a geração da resposta, a busca de imóveis por embedding, e o resumo com o follow-up. O caminho de volta desfaz a troca antes de o lead ler.

9.2 Os direitos do titular, na tela

DIREITO	ONDE	O QUE FAZ
Acesso (art. 18, II)	Chat, Meus dados, Baixar	JSON com tudo que o sistema guarda
Exclusão (art. 18, VI)	Chat, Meus dados, Excluir	Apaga lead, mensagens, agendamentos e memória
Revogação (art. 8o, §5o)	Chat, Meus dados, Não quero mais ser procurado	Desliga o follow-up sem apagar a conversa

Há também **retenção**: a memória da IA tem prazo de 180 dias, e a purga roda na subida do backend, apagando as duas representações dos leads vencidos. E há **transparência**: no primeiro turno, abaixo da caixa de consentimento, o lead lê que a conversa é respondida por IA e processada pelo Google Gemini. Esse aviso não tem caixa para marcar, porque é informação (art. 9o), não consentimento.

9.3 Os limites, declarados

POR QUE ESTA PARTE EXISTE

Declarar o limite antes de perguntarem vale mais que a proteção em si. Um sistema apresentado como maior do que é vira um problema no momento em que alguém testa.

A detecção é por expressão regular, não por modelo. Ela cobre bem o formato brasileiro comum e deixa passar nome em minúscula (“sou marcos”), nome estrangeiro fora do padrão, e endereço escrito de forma livre. Uma detecção mais agressiva mascararia bairro e nome de imóvel, e o agente perderia a informação de que precisa.

O mapa de apelidos fica no mesmo banco que o texto. Quem tem acesso ao arquivo

`backend/data/app.db` reverte tudo com uma consulta. Isso é proposital: a proteção é contra o terceiro que processa a conversa, não contra quem tem o banco.

Não há autenticação. O painel do corretor e o chat do lead são a mesma aplicação, no mesmo endereço. Duas coisas reduzem o risco: o identificador do lead deixou de ser sequencial e virou um hexadecimal de 16 dígitos, e a variável `VITE_MODO_CORRETOR=false` esconde as abas do painel. Esconder a navegação não é autenticação, e as rotas continuam existindo.

Os três cenários do desafio

O Tech Challenge pede três cenários. Aqui está como provocar cada um e o que observar, na lógica de comportamento e não de frase literal.

10.1 Cenário 1: lead novo, qualificado do zero

Como provocar. Clique em **Nova conversa**. Use uma das três frases de partida da seção 7.2 e conduza a conversa naturalmente até o fim, respondendo às perguntas do agente.

- ☐ A cada dado informado, uma etiqueta de “anotei” apareceu
- ☐ O painel do perfil foi preenchendo campo a campo
- ☐ A nota e a temperatura subiram ao longo da conversa
- ☐ Cards de imóvel apareceram assim que houve informação suficiente
- ☐ Os imóveis respeitaram o orçamento depois que você o informou
- ☐ Ao completar o perfil, o seletor de data apareceu sozinho
- ☐ Você confirmou o agendamento e ele apareceu na Agenda do painel
- ☐ O lead apareceu no Dashboard com a faixa de temperatura correta

Uma conversa completa tem de seis a oito mensagens, e consome de seis a oito requisições da cota diária.

10.2 Cenário 2: o lead volta, e a memória lembra

Como provocar. Com uma conversa em andamento, pressione **F5** para recarregar a página. Ou feche o navegador e abra de novo no mesmo endereço.

- ☐ O histórico da conversa voltou inteiro, na ordem certa
- ☐ O painel “O que já entendi” voltou **preenchido**, com os mesmos dados, a mesma nota e a mesma temperatura
- ☐ O mesmo identificador continua na barra da conversa, e não foi criado outro lead
- ☐ Ao continuar conversando, o agente não repetiu perguntas já respondidas

O segundo teste, mais forte: continue a conversa e informe um dado que contradiz o anterior. O agente deve absorver a correção e a etiqueta deve dizer **corrigi**, não “anotei”.

POR QUE ISSO É DIFÍCIL

O agente recebe apenas os dez turnos mais recentes. O que sustenta a memória além disso é a Parte 2, que guarda o perfil separado do histórico. Se tiver paciência para uma conversa longa, informe um dado no começo e pergunte sobre ele vinte mensagens depois: o perfil continua correto, mesmo o dado tendo saído da janela do modelo.

10.3 Cenário 3: o lead sumiu, e o follow-up retoma

Este cenário exige **duas condições** que não acontecem sozinhas numa sessão de teste, e é onde a maioria das pessoas trava.

Condição 1: consentimento

O follow-up não dispara sem consentimento registrado. A caixa “Quero que a imobiliária me procure” nasce **desmarcada** de propósito, porque sob a LGPD o aceite tem que ser um ato da pessoa, e caixa pré-marcada é exatamente o que a lei não admite.

Então, para este cenário: **marque a caixa antes de mandar a primeira mensagem** do lead de teste.

Condição 2: tempo de silêncio

A cadência exige 24 horas de silêncio para a primeira tentativa, ou 4 horas se a urgência do lead for alta. Ninguém numa sessão de teste está calado há um dia.

O projeto tem um script que mexe **só no relógio**, recuando a última mensagem do lead. Ele não inventa dado nenhum.

```
cd "D:\Projects\Pos\Fase 05\Tech-Challenge-FIAP-FASE5"

# ver os leads disponiveis
.venv\Scripts\python.exe docs\envelhecer-lead.py --listar

# envelhecer um lead especifico
.venv\Scripts\python.exe docs\envelhecer-lead.py lead-c03741594f3f4181

# envelhecer mais, para provocar a segunda tentativa
.venv\Scripts\python.exe docs\envelhecer-lead.py lead-c03741594f3f4181 --horas 80
```

Não precisa reiniciar o backend: a memória lê o estado do banco a cada consulta.

- ☐ O lead apareceu na seção **Precisam de atenção** do Dashboard
- ☐ O motivo da decisão está escrito em texto, como “24h de silêncio, tentativa 1 de 3”
- ☐ O botão de gerar texto produziu uma mensagem que **cita o que o lead disse**, e não um texto genérico
- ☐ Depois de envelhecer para 80 horas, a tentativa virou “2 de 3” e o tom mudou

As regras de quando NÃO enviar

Esta metade é a que separa follow-up de spam, e vale mostrar na apresentação:

- Sem consentimento registrado, não envia
- Se o lead pediu para parar, não envia nunca mais
- Se o lead respondeu depois do último follow-up, o contador zera
- Atingido o teto de três tentativas, encerra e devolve o lead ao corretor
- **Fora do horário comercial, adia.** O job não envia antes das 9h, depois das 20h, nem aos domingos

A cadência padrão é 24h, 72h e 168h. Para lead com urgência alta ela aperta para 4h, 24h e 72h, porque quem precisa mudar em duas semanas não espera três dias por uma resposta.

A JANELA ADIA O ENVIO, NÃO A CONSTATAÇÃO

A lista “Precisam de atenção” continua mostrando quem está devendo resposta a qualquer hora do dia. O que o horário comercial segura é a mensagem sair.

Quando algo não funciona

SINTOMA	CAUSA PROVÁVEL	O QUE FAZER
<code>Port 5173 is already in use</code>	Outro <code>npm run dev</code> ficou rodando	Use <code>npm run dev:3000</code> , ou encerre o processo antigo (seção 4.4)
Selo vermelho API offline	O terminal 1 caiu ou nunca subiu	Volte ao terminal 1 e rode <code>backend\run.py</code> . O selo volta ao verde em até 10 segundos, sem recarregar
Toda chamada falha, mas o backend está no ar	Front numa porta fora do CORS	Use 5173 ou 3000, ou acrescente a porta em <code>CORS_ORIGINS</code>
Selo âmbar sem você ter pedido	Chave ausente, inválida, projeto bloqueado ou cota esgotada	Rode <code>docs\verificar-chave.py</code> , que diz qual dos quatro
<code>agente: mock</code> com <code>modo_configurado: gemini</code>	A chave existe mas não funciona	Mesmo caminho acima. O campo <code>detalhe</code> traz o erro
Chat fica em “digitando” e não volta	Conexão travada com o Gemini	Espere até 90 segundos, que a requisição é cancelada com mensagem na tela. Ou clique em Nova conversa , que destrava o botão
Resposta “tive um problema técnico”	Falha no agente e também no mock	Veja o terminal 1. É o único caso em que os dois caminhos falharam
Nenhum imóvel aparece no chat	O sistema ainda não sabe nada do lead	Informe região, quartos ou faixa de preço. A busca só roda com pelo menos um deles
Imóveis param de aparecer no meio da conversa	Cota de embedding esgotada	O sistema cai para a busca lexical sozinho. Confira <code>rag.ultima_busca_ok</code> no <code>/health</code>
<code>rag.imoveis</code> é 0	A base não foi carregada	Confira se <code>shared\data\imoveis.json</code> existe, e reinicie o backend
“Precisam de atenção” sempre vazio	Falta consentimento, ou falta silêncio	Veja a seção 10.3. Os dois são exigidos
Follow-up não envia com lead elegível	Fora do horário comercial	O job não envia antes das 9h, depois das 20h nem aos domingos
Números do painel não batem	Banco diferente do que você imagina	Confira <code>database_url</code> no <code>/health</code>
Quer recomeçar do zero	Banco sujo de testes anteriores	Pare o backend, apague <code>backend\data\app.db</code> e suba de novo. Ele recria tudo e repopula os 140 imóveis

SINTOMA	CAUSA PROVÁVEL	O QUE FAZER
Erro de codificação no terminal	Console do Windows em cp1252	Rode <code>chcp 65001</code> antes de subir o backend

11.1 Onde olhar primeiro

Quando algo não fizer sentido, a ordem que resolve mais rápido é:

- 1 O **cabeçalho** da tela, que diz se a API está no ar e quem está respondendo.
- 2 O `/health`, que diz qual banco, quantos imóveis e qual modelo.
- 3 O **terminal 1**, onde o backend imprime cada requisição e cada erro.
- 4 O **console do navegador** (F12), para problemas que só acontecem na tela.

Limites conhecidos

Escrito com honestidade, porque um sistema apresentado como maior do que é vira um problema quando alguém testa.

Não há autenticação

Chat e painel são a mesma aplicação, no mesmo endereço. O projeto roda em localhost e foi pensado para isso. Publicar em qualquer lugar acessível por terceiros exige login de verdade, que está fora do escopo.

A detecção de dado pessoal é por expressão regular

Cobre e-mail, telefone, CPF e nomes declarados com inicial maiúscula. Deixa passar nome em minúscula, nome estrangeiro fora do padrão e endereço escrito de forma livre.

O mapa de apelidos fica no mesmo banco que o texto

A proteção é contra o terceiro que processa a conversa, não contra quem tem acesso ao arquivo do banco.

A cota gratuita é de 20 requisições por dia, por modelo

Isso dá duas ou três conversas completas por dia. O sistema degrada para o agente mock em vez de quebrar, e avisa na tela, mas a IA de verdade tem esse teto.

O banco é um SQLite de arquivo único

Suficiente para uma demonstração de um usuário, e tratado com WAL e transações curtas. Não foi pensado para vários corretores escrevendo ao mesmo tempo.

A extração é por regra, não por modelo

Ela é rápida, previsível e não gasta cota, e por isso mesmo tem limites conhecidos: entende as vinte regiões do catálogo, e não entende um bairro que não existe na base.

O catálogo tem 140 imóveis gerados sinteticamente

Eles são consistentes, com preço por metro quadrado coerente com o bairro e rentabilidade calculada, mas não são anúncios reais.

O sistema não fecha negócio

Ele qualifica e agenda. Falar de preço final, condição de pagamento e documentação é trabalho do corretor, e o agente foi instruído a devolver esses assuntos para a pessoa certa.

Ficha técnica

Tecnologias

CAMADA	TECNOLOGIA
Agente	Google Gemini (<code>gemini-2.5-flash</code>), SDK <code>google-genai</code>
Memória e RAG	Python puro, embeddings <code>gemini-embedding-001</code> com alternativa lexical
Backend	FastAPI, SQLAlchemy, SQLite, APScheduler
Frontend	Vite 7, React 19, TypeScript, Tailwind CSS 4

Constantes que valem conhecer

CONSTANTE	VALOR	ONDE
Janela de histórico enviada ao agente	10 turnos	<code>HISTORY_WINDOW</code>
Limiar de resumo da memória	20 mensagens	<code>SUMMARY_THRESHOLD</code>
Retenção da memória	180 dias	<code>DEFAULT_RETENTION_DAYS</code>
Cadência de follow-up	24h, 72h, 168h	<code>DEFAULT_CADENCE</code>
Cadência para urgência alta	4h, 24h, 72h	<code>URGENT_CADENCE</code>
Janela de envio	9h às 20h, sem domingo	<code>HORA_INICIAL</code> , <code>HORA_FINAL</code>
Limiar de lead quente	70 pontos	<code>HOT_THRESHOLD</code>
Limiar de lead morno	40 pontos	<code>WARM_THRESHOLD</code>
Teto por chamada ao Gemini	25 segundos	<code>TIMEOUT_MS</code>
Teto por requisição no front	90 segundos	<code>TIMEOUT_MS</code> (api.ts)
Intervalo do job de follow-up	30 minutos	<code>FOLLOWUP_INTERVAL_MINUTES</code>

Cobertura de testes

SUÍTE	QUANTIDADE	COMANDO
Partes 1 e 2	401 testes	<code>python -m pytest ai-core/tests ai-memory-rag/tests -q</code>
Backend, ponta a ponta	97 verificações	<code>python backend\tests\test_smoke.py</code>
Frontend, unitários	121 testes	<code>npm test</code>

SUÍTE	QUANTIDADE	COMANDO
Frontend contra a API no ar	12 testes	<code>npm run test:e2e</code>
Contrato entre front e API	75 verificações	<code>npm run test:contrato</code>

As duas suítes que falam com a API exigem o backend no ar. As demais rodam sozinhas.

Variáveis de ambiente

VARIÁVEL	PADRÃO	PARA QUE SERVE
<code>GEMINI_API_KEY</code>	vazio	A chave. Vazio significa modo mock
<code>GEMINI_MODEL</code>	<code>gemini-2.5-flash</code>	Modelo usado pelas duas metades
<code>DATABASE_URL</code>	SQLite em <code>backend/data/app.db</code>	Banco
<code>CORS_ORIGINS</code>	<code>localhost:5173,localhost:3000</code>	Origens liberadas
<code>FOLLOWUP_ENABLED</code>	<code>true</code>	Liga o job de retomada
<code>FOLLOWUP_INTERVAL_MINUTES</code>	<code>30</code>	De quanto em quanto o job acorda
<code>FOLLOWUP_USA_LLM</code>	<code>false</code>	Se o job gasta IA para escrever o texto
<code>SEED_ON_STARTUP</code>	<code>true</code>	Popular a base de imóveis na subida
<code>HISTORY_WINDOW</code>	<code>10</code>	Turnos enviados ao agente
<code>VITE_API_URL</code>	<code>http://localhost:8000</code>	Front: onde está a API
<code>VITE_MODO_CORRETOR</code>	<code>true</code>	Front: mostra ou esconde as abas do painel

Comandos de referência

Preparar, uma vez só

```
cd "D:\Projects\Pos\Fase 05\Tech-Challenge-FIAP-FASE5"
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r backend\requirements.txt

cd frontend
npm install
```

Subir, todo dia

```
# terminal 1
cd "D:\Projects\Pos\Fase 05\Tech-Challenge-FIAP-FASE5"
.venv\Scripts\python.exe backend\run.py

# terminal 2
cd "D:\Projects\Pos\Fase 05\Tech-Challenge-FIAP-FASE5\frontend"
npm run dev
```

No dia da apresentação

```
.venv\Scripts\python.exe backend\run.py --no-reload
```

Diagnóstico

```
.venv\Scripts\python.exe docs\verificar-chave.py
.venv\Scripts\python.exe docs\envelhecer-lead.py --listar
```

Rodar os testes

```
.venv\Scripts\python.exe -m pytest ai-core/tests ai-memory-rag/tests -q
.venv\Scripts\python.exe backend\tests\test_smoke.py

cd frontend
npm test
npm run test:e2e          # exige o backend no ar
npm run test:contrato     # exige o backend no ar
```

Recomeçar do zero

```
# pare o backend antes  
Remove-Item "D:\Projects\Pos\Fase 05\Tech-Challenge-FIAP-FASE5\backend\data\app.db"
```

Roteiro de cinco minutos

Para quem tem pouco tempo na banca, nesta ordem. Ele mostra o investidor, que é o perfil em que este projeto mais se diferencia.

MINUTO	O QUE FAZER, E O QUE APONTAR
1	Abra o chat. Mostre o cabeçalho com o selo verde e o aviso de transparência abaixo da caixa de consentimento. Marque a caixa.
2	Mande <code>quero investir em imóveis para alugar</code> . Mostre a etiqueta de intenção aparecendo como investimento , apesar da palavra “alugar” na frase. Responda ticket e retorno naturalmente.
3	Complete região, urgência e telefone. Mostre que ele nunca perguntou quantos quartos, e que o seletor apareceu falando em especialista , não em visita. Confirme o agendamento.
4	Vá ao Dashboard. Mostre o lead como QUENTE , abra a ficha e percorra os fatores da nota um a um. Mostre a próxima ação recomendada.
5	Volte ao chat, abra Meus dados , clique em Baixar e abra o JSON. Diga o limite em voz alta: “o mapa de apelidos fica no mesmo banco, então isso protege contra o Google, não contra quem tem o banco”.

SE A COTA TIVER ACABADO NO DIA

Não cancele a demonstração. O sistema roda inteiro em modo mock, com o selo âmbar avisando, e o roteiro acima funciona igual: as etiquetas, o painel, os imóveis, o score, o agendamento e a LGPD não dependem do Gemini. O que muda é a redação das respostas. Dizer isso à banca, antes que perguntem, demonstra a tolerância a falhas em vez de escondê-la.

Agente SDR Imobiliário, Tech Challenge FIAP, Fase 5. Este manual acompanha o repositório do projeto, e os documentos `docs/roteiro-de-teste-manual.md` (conferência item a item) e `docs/privacidade-e-limites.md` (o que sai do sistema, e até onde a proteção vai) aprofundam o que aqui está resumido.